Competitor Research Agent: Project Documentation

Nov 8, 2026 · @Surya

Overview

The Competitor Research Agent turns a company name into a verified, cited competitor brief in about 2 minutes, with a person approving the work at four points. It is built for Project 3A (Market Research Agent: Competitor Analysis) on the code track, with LangGraph, Tavily and Nebius Token Factory, and a Streamlit interface.

One-liner. My agent helps a product manager, analyst or founder research a company's competitive landscape in a Streamlit web app, replacing hours of manual searching across pricing pages, review sites and news. It identifies the company, finds its top 3 direct competitors, researches the company and each competitor, verifies every claim against its source and drafts a cited brief on its own, using 3 Tavily tools (Search, Map, Extract) and Nebius models. It hands off to a human to confirm an unclear company, approve the competitors, resolve flagged issues and approve the brief. I'll know it works when a user gets an approved brief in under 5 minutes with at least 90% of found claims passing verification.

Layer	Choice
Orchestration	LangGraph 1.2: `StateGraph`, `Send` for parallel branches, `interrupt` for human checkpoints, `SqliteSaver` checkpointer
Search and web data	Tavily Search (web and news), Map (site URLs), Extract (full page text)
Models	Nebius Token Factory: Qwen3-235B-A22B (strong), Qwen3-30B-A3B (fast), DeepSeek-V4.1-Flash (backup)
Interface	Streamlit app, plus a command-line runner
Storage	SQLite: checkpoints, search cache, budget ledger, competitor memory
Tooling	Python 3.12, uv, pytest (129 offline tests)

How the build maps to the Project 3A requirements:

Requirement	Where it lives
Agent 1 discovers the top 3 competitors with Tavily	Discovery agent: plans searches, ranks direct competitors, proposes the top 3
Agent 2 gathers fresh web and news data	Research agent: web and news search (90-day window), site mapping, page extraction
Agent 3 extracts pricing, features, positioning, news	Extraction agent: structured claims, each with an exact quote and a source
Orchestrator coordinates and compiles briefings	The LangGraph graph, plus the Synthesizer and Brief Writer
Added subagents	Scope Resolver, Verifier
Tavily as primary source, simple interface	Every web call goes through Tavily; Streamlit app
Sample output for 2 to 3 competitors	Three sample briefs below, each covering 3 competitors

Agent framework

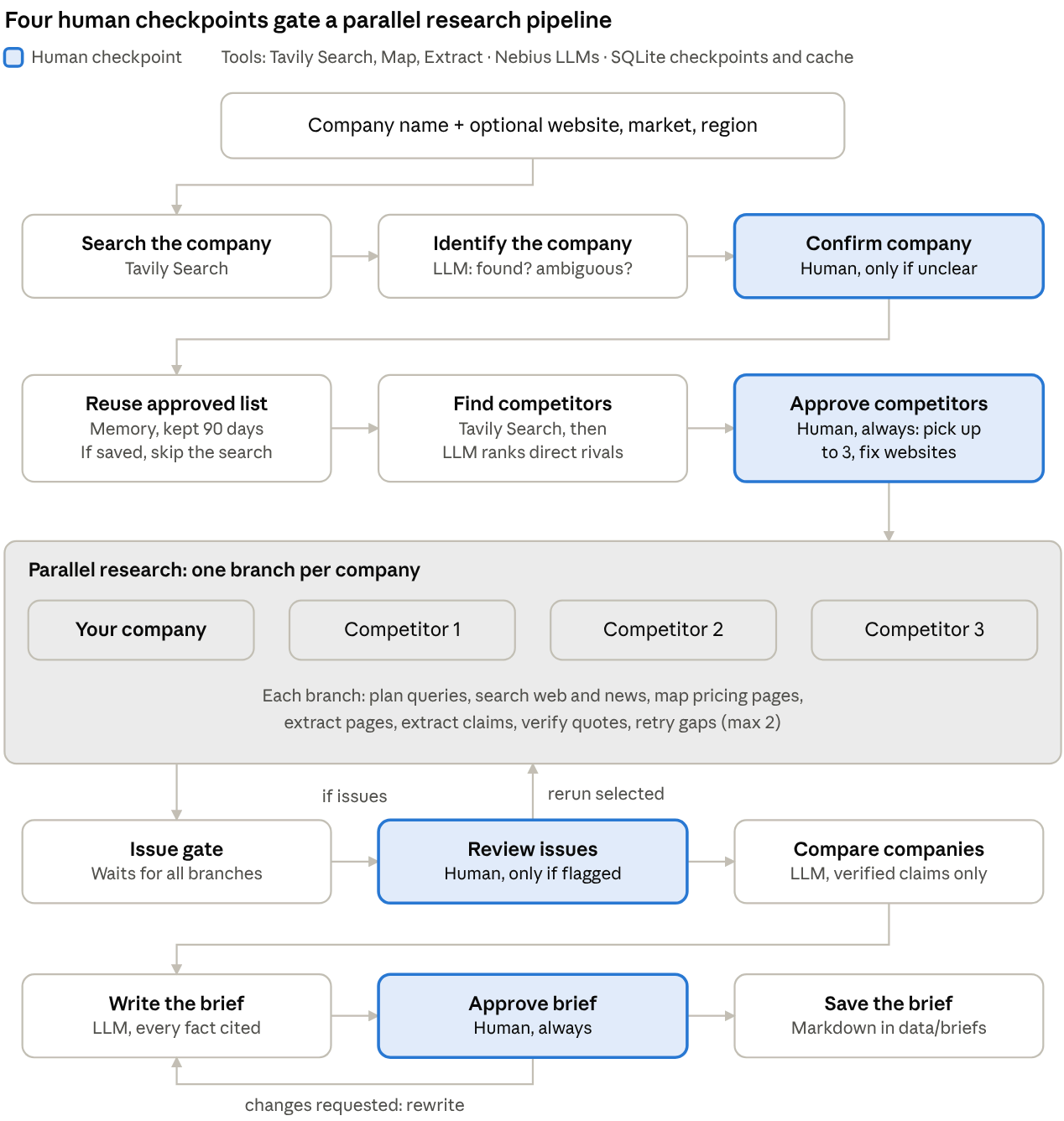
Every field of the handout's framework has a decision behind it; the sections after this one show how each is built.

Field	Decision
Agent goal	Takes a company name and produces a cited brief comparing it with its top 3 direct competitors.
Where people use it	A local Streamlit web app; a command-line runner for scripted runs and evaluation.
Steps, in order	1. Check the input. 2. Identify the company. 3. Find and rank competitors (or reuse ones approved before). 4. Research the company and each competitor in parallel. 5. Extract claims. 6. Verify claims and retry gaps. 7. Compare and write the brief. 8. Save it.
What it can do	Look-ups only: Tavily Search, Tavily Map, Tavily Extract, and Nebius model calls. Its only writes are local: saving the brief and its own run records.
What it remembers	Within a run: everything, checkpointed after each step so runs pause and resume. Across runs: approved competitors for 90 days, search results for 1 to 7 days, and every paid call in a budget ledger.
What it must never do	Invent a company, competitor, price or date; cite a source that does not support the claim; follow instructions found inside web pages; spend past the per-run budget.
Human in the loop	Four checkpoints: confirm an unclear company, approve the competitors (always), resolve flagged issues, approve the brief (always).
When something breaks	Retry, then change the query, then try another source, then ask the person, then deliver with the gap marked. One failed competitor never stops the others.

Field	Decision
How we know it worked	A brief the user approves, with at least 90% of found claims passing verification. Measured: 8 of 8 runs completed, 94% of claims verified.

Architecture

The orchestrator is the LangGraph graph itself: plain code fixes the order of steps, and LLMs make bounded judgements at specific nodes. Research runs in parallel, one branch for the company being analysed plus one per approved competitor, and four human checkpoints gate the decisions that shape the output.



System architecture · 6 stages, 4 human checkpoints, 4 parallel branches

Read top to bottom; highlighted boxes pause for a person. Two loops return work: a reviewer can rerun selected branches, and feedback on the brief sends it back for a rewrite.

Agent	Role	Node type
Orchestrator	The LangGraph graph: routing functions, fan-out with `Send`, checkpoints, interrupts	Control
Scope Resolver	Identifies which company the user means; reports “not found” rather than inventing one	Action + reasoning
Agent 1: Discovery	Reuses approved competitors or searches for candidates and ranks direct ones (same category, buyer, region)	Action + reasoning
Agent 2: Research	Plans queries, searches web and news, maps sites for pricing pages, extracts full pages	Reasoning + action
Agent 3: Extraction	Turns evidence into claims: pricing per plan, features, positioning, news, each with a quote and source	Reasoning
Verifier	Code checks (quote in source, source names the company, recency) plus LLM review (conflicts, wrong company, gaps)	Action + reasoning
Synthesizer and Brief Writer	Compare everyone against the company and write the cited brief from verified claims only	Reasoning

Reasoning nodes call an LLM and never call tools; action nodes call Tavily or run code and never call the LLM. Web content therefore cannot trigger an action, and every paid call passes through one place that checks the budget.

State management and orchestration

One shared run record lives in LangGraph state, and plain code controls the order of steps; the LLM is stateless and sees only the slice of state each node passes it. Anything that must survive a crash or be shared by parallel branches is written to SQLite immediately.

Layer	What it holds	Where	Lifetime
Run state	Company, constraints, candidates, approved competitors, evidence, claims, decisions, brief	LangGraph state	One run
Branch state	One company's queries, evidence, claims, retry count, gaps	Private to its parallel branch	One branch
Checkpoints	Run state after every step	`data/checkpoints.sqlite`	Until deleted
Search cache	Tavily results: news 24 hours, pages and site maps 7 days	`data/app.sqlite`	Until expiry
Budget ledger	Each paid call's reserved and actual cost	`data/app.sqlite`	Permanent
Competitor memory	Competitors approved per company, market and region	`data/app.sqlite`	90 days

How the orchestration works:

- **Fixed routing in code.** Router functions read the state and pick the next node; the LLM supplies judgements (rank these competitors, are these claims sufficient) but never chooses the path. A model mistake can change a claim, not skip the Verifier or the budget.
- **Parallel fan-out and fan-in.** After approval, `fan_out` sends one `Send` per company: the baseline company plus up to 3 competitors, 3 at a time. The issue gate runs once, after every branch finishes.
- **Merging by id.** Evidence and claims are dictionaries keyed by stable ids and merged by reducers, so branches never overwrite each other. A rerun replaces a claim with the same id and marks claims it no longer produces as superseded.
- **Budget shared across branches.** Every paid call reserves its worst-case cost in the SQLite ledger inside one transaction before it runs, then settles the actual cost. Parallel branches cannot jointly overspend the per-run limit (90 Tavily credits, $0.50 of Nebius).

- **Safe reloading.** The checkpointer only deserialises this project's own schema types.

Human in the loop

A person decides at four points: two always, two only when the agent is unsure. Research itself runs autonomously because it only reads; the decisions that shape the output, and saving the brief, wait for a human.

Checkpoint	When it pauses	The person's choices	Why here
Confirm company	Only if the name is ambiguous or no company was found	Use one of the options, search again with a corrected name or website, or cancel	A wrong company makes the whole run wrong
Approve competitors	Always	Select up to 3, correct a website, add a competitor, find competitors again (when they were remembered), or cancel	Highest-leverage decision, made before most of the budget is spent
Review issues	Only if a branch failed, a required field is missing, or claims conflict	Rerun selected companies, exclude claims from the brief, continue, or cancel	The person sees only what needs attention
Approve brief	Always (when a brief was written)	Approve, or describe changes and get a rewrite	Nothing is saved without sign-off

How pausing works:

- Each checkpoint is a LangGraph `interrupt()`. The run's state is checkpointed and the graph stops; the app reads the pending question from the checkpoint and shows its screen.
- The answer resumes the graph with `Command(resume=…)`, addressed to that interrupt's id. The run can wait for hours, survive an app restart, and be reopened from **Previous runs**.
- Answers are validated before resuming. An invalid name or website leaves the run paused and asks again.

- Every decision is stored in the run's `human_decisions` and overrides agent output: an excluded claim stays excluded even if a rerun extracts it again, and a corrected website is remembered for the next run.
- Feedback on a brief routes back only to the Brief Writer; a rerun routes back only to the named companies' research branches.

Failure handling

The agent degrades instead of guessing: retry, then change the query, then try another source, then ask the person, then deliver with the gap marked. A missing price is reported as `not_public` or `not_found`, never filled in.

Failure	How it's detected	Response
Junk input (e.g. `#@!$`)	Input checks before any spend	Rejected with a reason; nothing is spent
Company not found	Scope Resolver sets `found: false`	Ask the person: search again or cancel
Tavily rate limit, timeout, server error	429, timeout, 5xx	Retry up to 3 times with backoff; failed calls are not charged or cached
Bad Tavily key or plan limit	401, 403, 432, 433	Stop at once with a clear error
Model call fails or model retired	API error after the SDK's own retries	Switch to the backup model, logged
Model returns invalid JSON	Pydantic validation fails	One repair attempt with the error, then the backup model
Wrong competitor website	Site-only search returns nothing	Search the whole web, note "the domain may be wrong"; fix it at approval
Pricing page missing from search	No first-party pricing page	Map the site for pricing paths, try `<domain>/pricing`, extract with JavaScript rendering

Failure	How it's detected	Response
Claim not supported by its source	Quote not found in the source text	Excluded with the reason; prices whose figures all appear in the page are kept at low confidence
Claim about a different company	Source doesn't name the competitor as a word, or Verifier flags it	Excluded ("Planes" is not "Plane")
Stale data	News older than 90 days; prices older than a year	News and third-party prices excluded; old first-party prices flagged low confidence
Conflicting claims	Verifier compares claims for the same plan	Marked conflicting and sent to issue review
Gaps after research	Required field has no usable claim	Up to 2 targeted retries per company while budget remains
One branch crashes	Exception inside the branch	Caught; other branches finish; the person can rerun it
App crash or restart	Run has a next step but no pending question	Resume from the last checkpoint; cached searches are not paid for again
Budget reached	Ledger reservation refused	Stop spending, deliver what was verified
Instructions inside a web page	Treated as data by design	Reasoning nodes cannot call tools; the injection test in model selection was passed

Sample competitor briefs

Three briefs from runs made in the app on 2026-11-08, each comparing the company against the 3 competitors approved at review, with every fact cited to a source in the full brief. Figures are the system's findings as retrieved that day, not hand-checked.

Company	Competitors approved	Claims passing verification	Tavily credits	Nebius cost	Brief file
AdvancedMD	Tebra, RXNT, OmniMD	47 of 49 (16 about AdvancedMD)	21	$0.034	`run-c2d7e891.md`
Global Payments	Stripe, Adyen, Braintree	46 of 49 (11 about Global Payments)	47	$0.039	`run-f93cd43d.md`
Nike	Adidas, Puma, HOKA	67 of 71 (29 about Nike)	43	$0.025	`run-591fc6d4.md`

Each run paused twice for a person, to approve the competitors and to approve the brief; none needed issue review. The Verifier excluded 9 claims across the three, for quotes missing from their source, news older than 90 days, or a claim about a different company.

Brief 1: AdvancedMD vs Tebra, RXNT and OmniMD

RXNT undercuts AdvancedMD's entry price and OmniMD includes AI scribing at no extra cost, while AdvancedMD relies on third-party AI documentation tools.

Company	Pricing found	Positioning
AdvancedMD	$130 to $1,070 per provider per month by specialty; $0.87 to $2.17 per claim; billing services 4 to 8% of collections	Independent, group and specialty practices across 70+ specialties
RXNT	$126 (EHR) and $335 (full suite) per provider per month; $665 a year for e-prescribing alone	Solo and small practices (2 to 9 providers)
Tebra	$99 to $799 per provider per month by plan; AI add-ons $99 to $149	Complete operating system for independent practices
OmniMD	Custom, by providers and modules; AI scribe included	Multispecialty and group practices consolidating systems

- **Implications:** AdvancedMD's per-claim pricing suits high-volume practices that per-provider plans do not; its NeuroStar TrakStar integration gives it an edge in mental health and TMS care.

- **Recent news:** AdvancedMD's TrakStar integration (2026); RXNT on TIME's World's Top HealthTech Companies list for a second year (2026); Tebra's AI Note Assist passing 500,000 notes.

Brief 2: Global Payments vs Stripe, Adyen and Braintree

Global Payments is the only one of the four with no public pricing, as it repositions around Worldpay and its Genius AI assistant.

Company	Pricing found	Positioning
Global Payments	Not public for any product line	Payment technology for businesses of all sizes, now in SMB, Enterprise and Platforms segments
Stripe	2.9% + 30¢ per card payment; ACH 0.8% capped at $5	API-first payments and billing for developers and SaaS
Adyen	Regional fees, e.g. $0.13 + 3.3% + $0.10 in North America; Interchange++ globally	Full-stack platform for global enterprises
Braintree	2.89% + $0.29 for cards and wallets; Venmo 3.49% + $0.49	End-to-end checkout including PayPal and Venmo

- **Implications:** opaque pricing is a disadvantage against three rivals that publish rates; the Worldpay integration is Global Payments' route to compete with Stripe and Adyen in enterprise and cross-border payments.
- **Recent news:** Global Payments rolling out Genius to 30+ bank partners (Oct 2026) after buying Worldpay for $22.7 billion and selling its issuer business to FIS for $13.5 billion; Adyen's Toast partnership in the US (Aug 2026) and Flatpay partnership in Europe (Sep 2026); Stripe's tools for Asian businesses expanding abroad (Aug 2026). No recent Braintree news was found.

Brief 3: Nike vs Adidas, Puma and HOKA

Nike has the widest price range and the most quantified technology claims, while Adidas has momentum in performance and HOKA keeps growing in cushioned running.

Brand	Prices found	Positioning
Nike	$30 (kids' slides) to $235 (Air Jordan 11 Retro); Pegasus Plus 2 running shoe $170	Empowering people to achieve more; broad but personal appeal
Adidas	$68 (on sale) to $120; Supernova Rise 3 running shoe $98	Sports and fitness enthusiasts, youth, fashion-led buyers
Puma	$65 (kids) to $260 (Deviate NITRO Elite 4); MagMax NITRO 2 $180	Sport, fashion and lifestyle for millennials and Gen Z
HOKA	Up to $250; Kawana 3 $112 (was $140)	Maximalist cushioning for elite and casual runners

- **Implications:** Nike's injury-reduction data (React Infinity Run, 52% fewer injuries in a 12-week study) and lower-carbon ReactX foam support health and sustainability positioning; Puma's $260 NITRO Elite and HOKA's cushioning challenge Nike in elite and long-distance running.
- **Recent news:** Nike reorganized into Americas, APGC and EMEA (Oct 2026) and joined The Nature Conservancy's microfibre coalition (Oct 2026); Adidas Performance revenue up 39% in Q2 2026; HOKA revenue up 7.7% in Q2 2026.
- **Note:** consumer brands price per product, not per plan, so this brief samples prices rather than listing a price list.

Evaluation and results

All 8 evaluation runs produced a brief, every proposed competitor matched the reference lists, and 94% of found claims passed verification, at about 96 seconds, 30 Tavily credits and $0.03 of Nebius usage per run.

Model selection. `evals/model_selection.py` scored 6 Nebius models on three tasks taken from the pipeline (extracting claims from evidence with a planted prompt injection, judging evidence, ranking competitors), each run twice on fictional companies so answers could be checked exactly. The whole test cost $0.02.

Role	Model	Price per 1M tokens (in / out)	Why
Strong: ranking, extraction, verification, writing	Qwen3-235B-A22B-Instruct-2507	$0.20 / $0.60	Only low-cost model perfect on all three tasks, and the fastest
Fast: planning searches, identifying the company	Qwen3-30B-A3B-Instruct-2507	$0.10 / $0.30	Perfect on ranking, cheapest; missed statuses on extraction, so extraction moved to the strong model
Backup: any role on failure	DeepSeek-V4.1-Flash	$0.30 / $1.20	Perfect scores, different model family from the primaries

Pipeline evaluation. `evals/run_evals.py` ran the full pipeline on 8 companies across markets and one region, approving each checkpoint automatically. It ran before the company baseline and Tavily Map were added.

Metric	Result
Runs completed with a brief	8 of 8
Proposed competitors in the reference lists	24 of 24
Competitors with pricing, features and positioning	22 of 24
Found claims that passed verification	261 of 278 (94%)
Runs that needed issue review	3 of 8
Mean time, Tavily credits, Nebius cost per run	96 s, 30, $0.029

Companies: Linear, Notion, Figma, HubSpot, Zoom, Mailchimp, Shopify, and Razorpay (India). Claims were excluded mainly for news older than 90 days (9) and quotes missing from their source (7); one was about a different company. The reference lists of expected competitors were written for this project, so 24 of 24 measures agreement with them rather than an independent benchmark, and prices were not checked by hand. The full report lists every verified price with its source for spot checks.

How it was built

The system was designed and built in conversation with Claude Code over eight milestones, each ending with tests and a live run, and most fixes came from live runs that the offline tests could not catch.

Datasets and sources

- **Live web data** through Tavily: no training data or fixed dataset. Every claim in a brief comes from a page retrieved during the run.
- **Model-selection fixtures** (`evals/model_selection.py`): fictional companies with known answers, including a planted prompt injection.
- **Evaluation set** (`evals/companies.yaml`): 8 real companies with reference lists of direct competitors.
- **Offline test fakes**: canned LLM and Tavily responses for 129 tests that cost nothing to run.

Prompts used while vibe coding

The build was driven by plain-language prompts to Claude Code, in roughly this order:

1. “How would you go about designing and implementing this agentic system? Assume you are a market research analyst. Explain where human in the loop comes in, how state is maintained and orchestrated, and what fallback handles failures.”
2. “I have Nebius Token Factory credits for LLM usage.” and “Who is orchestrating the agents in this flow?”
3. “Let's go with graph and limited LLM decisions.”
4. “Certification project. Top 3 should be direct competitors. I have $25 each of Nebius and Tavily credits.”
5. “Explain how state is managed”, “how are the parallel branches triggered?”, “where do SQLite, cache and other memories come in?”, “are there action nodes and reason nodes?”
6. “Create the project using uv.”
7. “What if a user enters junk values?” then “add the website and fix issues 1 to 4”.
8. “Also account for a company name, domain area and region, if a user decides to enter those.”
9. “Complete all the milestones without my approval; by the time I come back I want to start testing the app.”

10. “Yes, add the company I search for as a baseline”, “Tavily may find more pricing information than just online search”, “cache competitor data so the next run pulls the same competitors”.

Iterations

Problem found	Found by	Change made
Qwen gave every claim the same status in Nebius JSON-schema mode	Live run, then a direct test of four output modes	Switched to JSON mode plus Pydantic validation; added status checks to the model test
Fast model missed an unpublished Enterprise price	Rerun of the model test	Extraction moved to the strong model (+$0.002 per company)
Grouped citations like [C1, C6] missing from Sources	Reading the first brief	Citation parser reads groups
Planner put “2024” in news queries and found other companies named Coda	Live run	Planner gets today’s date; queries must name the competitor
Discovery guessed Plane’s site as useplane.com; Plane got 0 evidence	Live run	Name forced into every query; site-only search falls back to the whole web; domain editable at approval
A “Plane” price came from “Planes”	Live brief	Verifier identity check (whole-word name match)
GitHub’s Free, Team and Enterprise prices flagged as conflicting	Live run	Claims store the plan or feature name
An empty answer resumed nothing	Offline test	Answers addressed to the interrupt’s id
Plane’s $6 extracted as “$66”, so correct prices were excluded	Live run	Prices whose figures appear in the page are kept at low confidence

Problem found	Found by	Change made
A 2022 press-release price passed as high confidence	Evaluation report	Old first-party prices flagged low confidence
AdvancedMD's own pricing absent from its brief	User testing	Company researched as a baseline; Tavily Map finds pricing pages
Repeat runs proposed different competitors	User request	Approved competitors remembered for 90 days

Learnings

- **Live runs find what fakes cannot.** Every serious bug above came from real pages: odd dates, JavaScript prices, shared company names.
- **Check the model's output format, not just its answers.** The same model was right in JSON mode and wrong in schema mode.
- **Code should own the rules.** Prompts asked the planner to name the competitor; code now guarantees it.
- **Exclusions need a reason the user can see.** Showing why a claim was dropped made each failure easy to trace and fix.
- **Human checkpoints are cheapest early.** Approving competitors before research saves most of the budget when Discovery is wrong.

Running the project and limitations

The app runs locally with three commands and serves only this machine, because it spends the user's API credits.

```
uv sync
uv run python -W ignore -m competitor_research_agent.startup
uv run streamlit run app.py
```

The startup check confirms both API keys, that the configured models exist, and that each returns valid structured output. The app opens at http://localhost:8501: enter a company, optional website, market and region, then follow the review screens. Results

show in four tabs (Brief, Claims, Sources, Run details); approved briefs are saved to `data/briefs/<run id>.md`. A run costs about 45 to 50 Tavily credits and $0.04 to $0.05 of Nebius usage, under a per-run cap of 90 credits and $0.50.

Known limitations:

- Prices were not hand-checked against live sites; the evaluation report lists each with its source for spot checks.
- Discovery can still propose a wrong website for a competitor; the run works around it, and the user can correct it at approval.
- Some sites block extraction (Coda's pricing page did), leaving pricing `not_public` or `not_found`.
- With no region set, a site may serve local prices (Zoho CRM appeared in rupees).
- Single user with local SQLite storage; no scheduled monitoring of competitors over time.
- The 8-company evaluation predates the company baseline and Tavily Map, and should be rerun for current figures.